

Breadth-First Search

Queues, shortest routes, and DFS vs. BFS

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 12 Thursday, October 8, 2026

Goals for Today

- Explain why recursive DFS can return a route that is **not the shortest**.
- Describe BFS: explore a maze in **layers**, using a **queue**.
- Trace BFS by hand: the queue, the visited marks, and the **parent table**.
- **Rebuild the route** by following parent links from the end back to the start.
- Compare DFS and BFS: route, container, time and space.
- See how DFS_Solver and BFS_Solver are used through a Maze_Solver*.

Where We Are

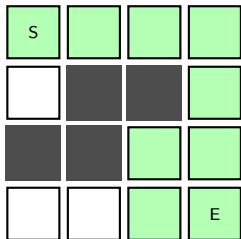
Date	What
Today, Thu 10/8	Class 12: BFS. Exercise 5 is due.
Tue 10/13	No class (Tuesday follows the Monday schedule)
Thu 10/15	Class 13: Midterm Review
Project 2	Task A (recursive DFS) is Class 11. Task B (BFS)

Recap from Class 11

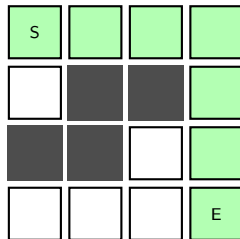
Recursive DFS asks “is there a path from here?”, marks each cell visited, and tries left, right, up, down. The **call stack** remembers the current route. Backtracking is a **return**.

DFS Found a Route. Is It the Shortest?

The 4×4 maze from the Project 2 handout. Black cells are walls; $S = (0, 0)$, $E = (3, 3)$.



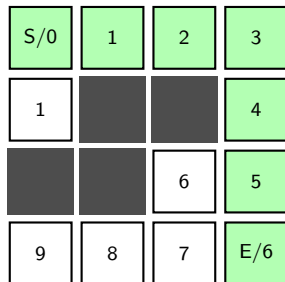
DFS: 9 cells



BFS: 7 cells

DFS commits to a branch and returns the **first** route it finds. When we need the route with the **fewest cells**, we need a different search order.

The BFS Idea: Explore in Layers



Numbers: steps from S . Green: the route.

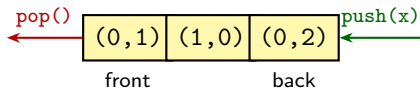
Picture a stone dropped in a pond. The ripples reach every cell 1 step away, then every cell 2 steps away, then 3, and so on.

- Layer 0: S .
- Layer 1: $(0, 1)$ and $(1, 0)$.
- Layer 2: $(0, 2)$. Layer 3: $(0, 3)$
- The **first time** a ripple reaches E , it got there in the fewest steps.

Question: what data structure hands cells back in the order they were discovered?

The Answer: a Queue

- A **queue** is first in, first out (Class 10). Cells discovered earlier are explored earlier, so all cells at distance d are explored before any cell at distance $d + 1$.
- DFS uses a **stack** (the call stack): the *newest* cell is explored first, so the search runs deep.
- BFS uses a **queue**: the *oldest* waiting cell is explored first, so the search spreads out.



In C++: `std::queue<Coord>` with `push()`, `front()`, `pop()`, `empty()`.

BFS in Pseudocode

```
getPath():  
  if start or end is not open: return empty stack  
  queue q; mark start visited; q.push(start)  
  while q is not empty:  
    cur = q.front(); q.pop()  
    if cur is the end: stop searching  
    for each neighbor of cur (any fixed order):  
      if neighbor is open and not visited:  
        mark neighbor visited      // when ENQUEUED  
        parent[neighbor] = cur     // who discovered it  
        q.push(neighbor)  
  if the end was never reached: return empty stack  
  walk from the end back to the start by parents,  
  pushing each cell on a stack (start ends on top)
```

- Two tables, **visited** and **parent**, have the same shape as the maze. The **queue** holds the cells waiting to be explored.
- Unlike DFS, **any** neighbor order is fine: every shortest route has the same number of cells.

Why Mark Cells When They Are *Enqueued*?

An open 2×2 grid, $S = (0, 0)$. Cell $(1, 1)$ is a neighbor of both $(0, 1)$ and $(1, 0)$.

Mark when dequeued (wrong)	Mark when enqueued (right)
Dequeue S : enqueue $(0, 1)$, $(1, 0)$	Dequeue S : mark and enqueue $(0, 1)$, $(1, 0)$
Dequeue $(0, 1)$: $(1, 1)$ not marked yet, enqueue it	Dequeue $(0, 1)$: mark and enqueue $(1, 1)$
Dequeue $(1, 0)$: $(1, 1)$ <i>still</i> not marked, enqueue it again	Dequeue $(1, 0)$: $(1, 1)$ is already marked, skip
Queue holds $(1, 1)$ twice	Queue holds $(1, 1)$ once

Rule

Mark a cell visited the moment you put it in the queue. Then each cell enters the queue **at most once**, and its parent is set once and never overwritten.

BFS Step by Step: the Maze from Slip Question 3

	0	1	2
0	S		
1			
2	T		

Same maze as Class 11. DFS (left, right, up, down) entered 7 cells before reaching *T*.

BFS uses the same neighbor order here, so the steps are easy to compare.

Watch:

- the **queue**: front on the left;
- the **parent table**: who discovered each cell.

Orange: just dequeued. Yellow: in the queue. Blue: explored. White: not discovered. Green: final route.

BFS Step by Step: Step 1 of 5

	0	1	2
0	S		
1			
2	T		

Queue (front $\rightarrow$ back)

(0,0)

Parent table

(empty)

Start: mark $S = (0,0)$ visited and enqueue it. The queue holds the cells we have discovered but not yet explored.

BFS Step by Step: Step 2 of 5

	0	1	2
0	S		
1			
2	T		

Queue (front $\rightarrow$ back)

(0,1)	(1,0)
-------	-------

Parent table

$\text{parent}(0,1) = (0,0)$

$\text{parent}(1,0) = (0,0)$

Dequeue (0,0). Neighbors: left outside; right **open**: mark, parent = (0,0), enqueue; up outside; down **open**: mark, parent = (0,0), enqueue.

BFS Step by Step: Step 3 of 5

	0	1	2
0	S		
1			
2	T		

Queue (front $\rightarrow$ back)

(1,0)	(0,2)
-------	-------

Parent table

parent(0,1) = (0,0)

parent(0,2) = (0,1)

parent(1,0) = (0,0)

Dequeue (0,1). Neighbors: left visited; right **open**: mark, parent = (0,1), enqueue; up outside; down wall.

BFS Step by Step: Step 4 of 5

	0	1	2
0	S		
1			
2	T		

Queue (front $\rightarrow$ back)

(0,2)	(2,0)
-------	-------

Parent table

parent(0,1) = (0,0)

parent(0,2) = (0,1)

parent(1,0) = (0,0)

parent(2,0) = (1,0)

Dequeue (1,0). Neighbors: left outside; right wall; up visited; down **open**: mark, parent = (1,0), enqueue. The end is discovered, so we can stop.

BFS Step by Step: Step 5 of 5

	0	1	2
0	S		
1			
2	T		

Queue (front → back)

(0,2)	(2,0)
-------	-------

Parent table

parent(0,1) = (0,0)

parent(0,2) = (0,1)

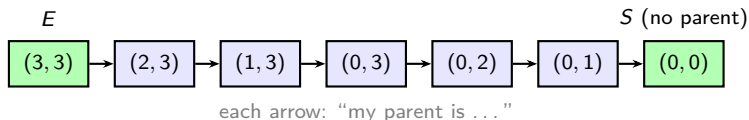
parent(1,0) = (0,0)

parent(2,0) = (1,0)

Rebuild the route: start at the end and follow the parents back: $(2,0) \leftarrow (1,0) \leftarrow (0,0)$. Pushing each cell on a stack as we go leaves the **start on top**.

Rebuilding the Route: Parents Form a Linked List

On the 4×4 maze, BFS records one parent per discovered cell. Start at E and follow the parents:



- This is a **singly linked list**: each cell's "next" is its parent, and the list ends at S .
- We reach the cells from end to start, but the answer must have the **start on top**. A **stack** reverses the order: push E first, push S last.
- Same pattern as printing a string backward: the last thing seen comes out first.
- If E was never discovered, there is no route: return an **empty stack**.

DFS vs. BFS: Side by Side

	DFS (recursive)	BFS
Container	Call stack	<code>std::queue</code> (explicit)
Search order	Deep along one branch, then back up	Layer by layer
Route found	First one found; 9 cells here	A route with the fewest cells; 7 cells here
Backtracking	Yes: a return	None: nothing is ever undone
Route is built by	Unwinding the calls (answer stack)	Following parents from <i>E</i> to <i>S</i>
Neighbor order	Matters: it decides the route	Any order: same length
4×4 example	Enters 11 cells	Marks 9 cells (including <i>S</i> and <i>E</i>)

Time and Space

Let n be the total number of cells in the maze.

	DFS	BFS
Time	$O(n)$: each cell is entered once, with at most 4 neighbor checks	$O(n)$: each cell is enqueued and dequeued once, with at most 4 neighbor checks
Extra space	Visited table $O(n)$; call stack up to $O(n)$ deep in a long winding maze	Visited and parent tables $O(n)$; queue holds the current frontier, at most $O(n)$

Take-away

Both visit each cell at most once, so both are linear. The real difference is **which route you get**, not how fast you get it.

Which One Should I Use?

Use BFS when...

- you need the route with the **fewest steps** (unweighted);
- “what is the distance from S to E ?”

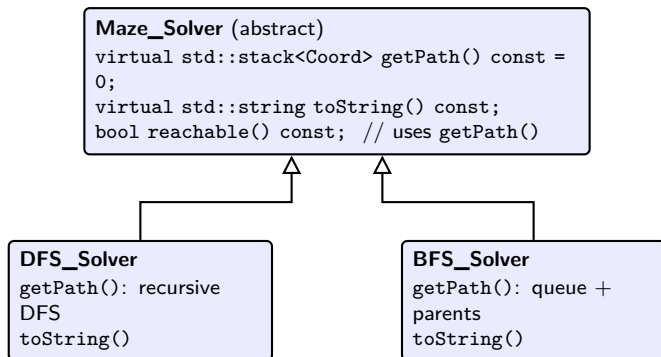
Use DFS when...

- **any** route is enough, or you want to explore *every* possibility;
- the problem is naturally recursive and you try choices and **undo** them.

Looking ahead

After the midterm, Project 3 (the Make 24 puzzle) tries choices, recurses, and backs up when a choice fails: recursion and **backtracking**, the DFS pattern in a new setting.

One Interface, Two Solvers



`reachable()` is written **once** in the base class:

`!getPath().empty()`. It works for both solvers because `getPath()` is **virtual**.

Calling Through a Base-Class Pointer

```
DFS_Solver dfs(room, start, end);
BFS_Solver bfs(room, start, end);
Maze_Solver* solvers[] {&dfs, &bfs};

for (const Maze_Solver* curr : solvers) {
    std::cout << curr->toString() << '\n';
    // reachable() and getPath() run the version that
    // belongs to the REAL object, not the pointer type
}
```

- The pointer type is `Maze_Solver*`, but the object is a `DFS_Solver` or a `BFS_Solver`. A **virtual** call is decided at **run time** by the real object.
- The loop does not know, or care, which search it is running. This is Task D of Project 2.
- Without `virtual`, the call would always run the base-class version.
- The base class declares a **virtual destructor** so that deleting through a base pointer is safe.

Project 2, Task B: Checklist

- Use `std::queue<Coord>`. Mark cells visited when **enqueued**.
- Keep a **parent** table with the same shape as the maze (rows may have **different lengths**).
- Stop when the end is discovered (or dequeued): the route length is the same.
- Rebuild the route by following parents from the end to the start, pushing each cell on a `std::stack<Coord>` so the **start is on top**.
- Edge cases: start or end is a wall or outside the maze; empty maze; start equals end (one-cell route); no route $\Rightarrow$ **empty stack**.
- Check with `valgrind`: no leaks.

Summary

- **DFS** goes deep and returns the first route it finds.
BFS spreads out in layers and finds a route with the **fewest cells**.
- BFS uses a **queue**; mark a cell when it is **enqueued**; record a **parent** for each cell.
- The route is rebuilt by following **parents** (a linked list) from the end to the start; a **stack** puts the start on top.
- Both searches are $O(n)$ time; the difference is the **route**, not the speed.
- `DFS_Solver` and `BFS_Solver` share one interface through a virtual `getPath()`.

Quick Check

- 1 In the slip-3 maze, what is in the queue right after $(0, 1)$ is dequeued?
- 2 Why would marking cells only when dequeued put a cell in the queue twice?
- 3 BFS found E through the parents $(3, 3) \rightarrow (2, 3) \rightarrow \dots$. Why does the answer stack have the *start* on top?
- 4 Which of DFS or BFS can return a route that is not the shortest, and why?
- 5 What does `virtual` change in `curr->toString()`?